COMP 4300
Intro to Game Programming

Lecture 4
Entities, Components, Systems
Vec2 Class

ECS Game Programming

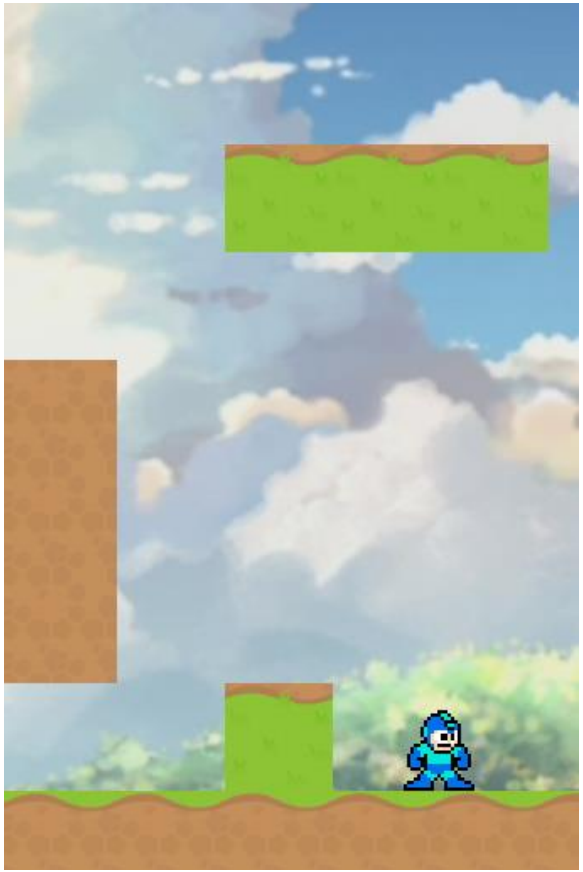
- Why use ECS for games?
- ECS = Entities, Components, Systems
- Entity: Any object in the game
  - Player, Platform, Tile, Bullet, Enemy
- ECS is actually a software design paradigm
- Let’s first look at Object-Oriented design

Important Notes

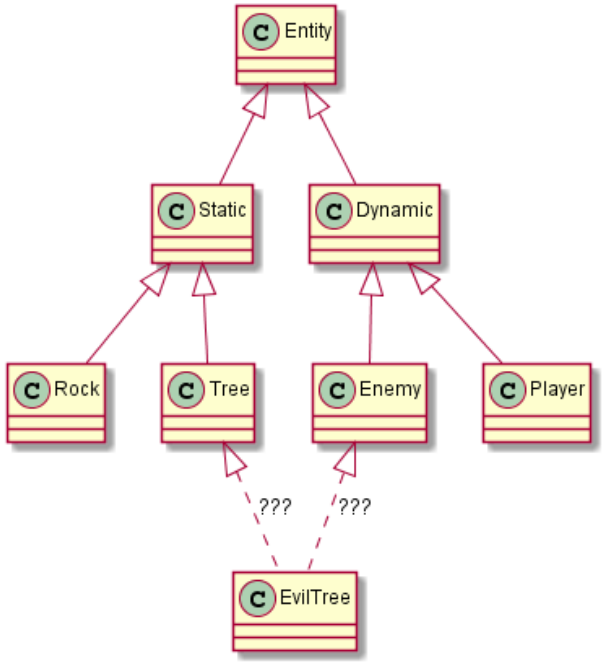
- Large AAA games can and have been made with Object Oriented Design for years
- I have decided to teach ECS because in my opinion it is better for this course, and will allow us to do many things quite easily
- Some agree with using ECS and some prefer OOD, neither is 100% provably best

Object-Oriented

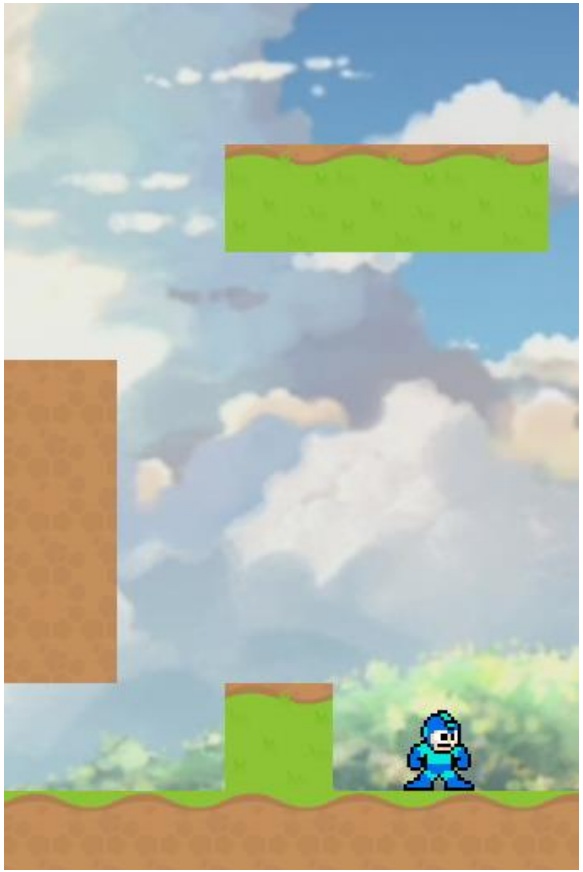
- OOP / OO Design helps us manage data / code
- Entity: any object in game
  - Each of them have (x,y) pos
  - Entity used as base-class
  - Tile, Player, inherit from it



Object-Oriented

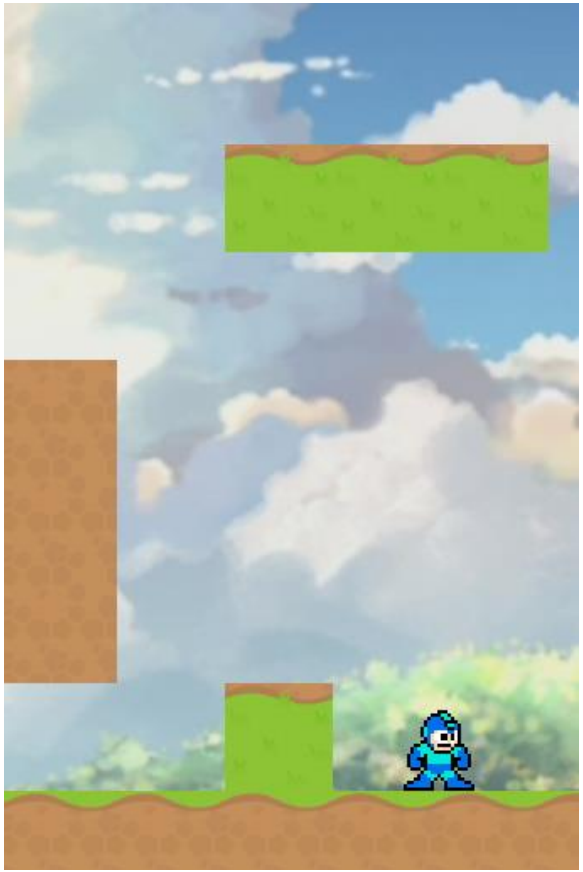


Class Diagram from: https://www.gamedev.net/articles/programming/general-and-gameplay-programming/understanding-component-entity-systems-r3013/

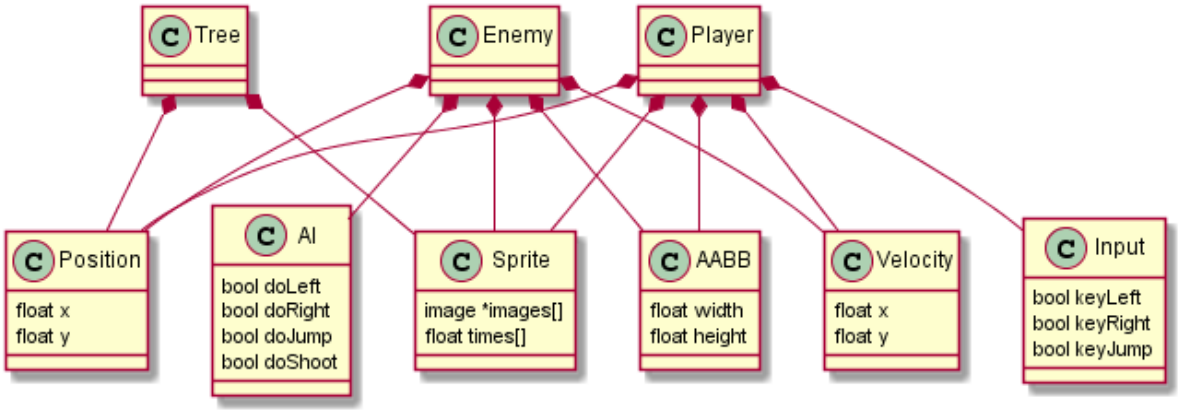


Entity-Component

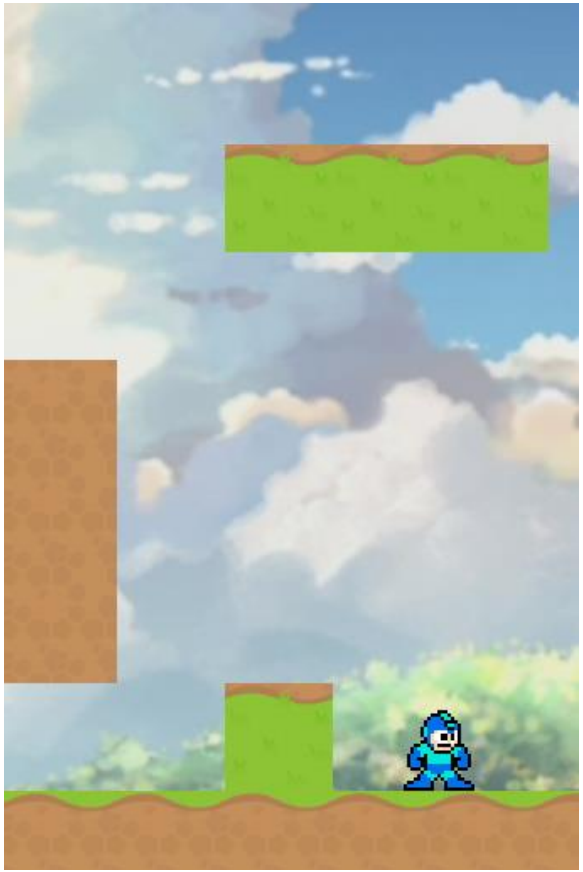
- Entity: object in game
- Component: properties that can attach to Entities
- Component examples:
  - Position, Speed
  - Bounding Box
  - Health, Weapon, Damage



Entity-Component



Class Diagram from: https://www.gamedev.net/articles/programming/general-and-gameplay-programming/understanding-component-entity-systems-r3013/



ECS Game Programming

- ECS uses Composition-based design
- Entity: Anything object in the game
  - Player, Platform, Tile, Bullet, Enemy
- Component: Properties attached to entities
  - Position, Texture, Animation, Health, Gravity
  - Components are PURE DATA
- Systems: Code / logic that drives behavior
  - Movement, Rendering, Sound, Physics

ECS Example Entity/Component

- Player
  - Pos, Speed, BBox, Sprite, Health, Gravity, Input
- Enemy
  - Pos, Speed, BBox, Sprite, Health, AI
- Bullet
  - Pos, Speed, Angle, BBox, Sprite, Damage, Lifespan
- Tile
  - Pos, BBox, Sprite

Important Notes

- This is not the only way to implement ECS
- The architecture in this course is a balance between efficiency, ease of use, and ease of learning for an undergraduate class
- We will make improvements as we go and at the end of the course discuss how we would optimize things if making a real game

ECS Example System

```
// movement system
for (e : entities) { e.pos += e.speed; }
// collision system
 for (b : bullets)
    for (e : enemies)
      if (Physics:IsCollision(b, e))
        e.health -= b.damage
        b.destroy();
// rendering system
for (e : entities) { window.draw(e.sprite, e.pos); }
```

Engine Architecture

```
GameEngine
  > Scene
    > Systems
    > EntityManager
      > Entity
        > Component
```

What is a Component?

- An ECS Component is just DATA!
  - (Maybe some logic in the constructor)
  - No helper functionality within components
- A Component class has some intuitive meaning to an Entity which contains it
  - Ex: Position, Gravity, Health, etc
- How to implement them in C++?

What is an Entity?

- Entity = any object in the game
  - Usually, any object with a position
- No unique functionality, typically just stores a number of Components
- Stores at most 1 of each component type
- How Components are stored and used within an Entity can be a complex topic

Component Storage Options

1. Since there will be at most one of every component, store a variable for each component type.
2. Store a single collection of Components. We can then use addComponent / getComponent functions for more generality than individual variables

1. Component Variables

```
class Entity
{
public:
    CTransform  cTransform;
    CName       cName;
    CShape      cShape;
    CBBox       cBBox;
    Entity() {}
}
```

1. Component Variables

```
Entity e;
e.cTransform = CTransform(Vec2(100, 200));
e.cName      = CName(“Red Box”);
e.cShape     = CShape(args);
if (e.cTransform.exists) { ... }
// memory safe, contiguous memory, no heap usage
// directly accessing variables is undesirable
   - requires knowledge of internal structure of Entity class
   - unsafe, allows users to directly modify vars
// Entity ‘contains’ component check requires another variable
   - also requires knowing that variable exists
```

2. Component Raw Pointer Vars

```
class Entity
{
public:
    CTransform * cTransform = nullptr;
    CName      * cName      = nullptr;
    CShape     * cShape     = nullptr;
    CBBox      * cBBox      = nullptr;
    Entity() {}
    ~Entity() {} // must free memory
}
```

2. Component Raw Pointer Vars

```
Entity e;
e.cTransform = new CTransform(Vec2(100, 200));
e.cName      = new CName(“Red Box”);
e.cShape     = new CShape(args);
if (e.cTransform) { ... }

// Entity contains component check slightly easier
// memory leak, must check and delete
   - makes this method very cumbersome and unsafe
// heap memory usage makes it slow as well
// direct variable access undesirable
```

3. Component Smart Pointers

```
class Entity
{
public:
    std::shared_ptr<Ctransform> cTransform;
    std::shared_ptr<CName>      cName;
    std::shared_ptr<CShape>     cShape;
    std::shared_ptr<CBBox>      cBBox;
    Entity() {}
}
```

3. Component Smart Pointers

```
Entity e;
e.cTransform = std::make_shared<Ctransform>(Vec2(100, 200));
e.cName      = std::make_shared<CName>(“Red Box”);
e.cShape     = std::make_shared<CShape>(args);
if (e.cTransform) { ... }
// memory safe, shared ptr implements RAII
// Entity contains component check is still easy
// slower, smart pointer overhead
// still slow due to heap memory usage
// longer syntax, pain to type every time
// still allows direct memory access
```

4. Component Container

```
class Entity
{
    std::vector<Component> m_components;
public:
    Entity() {}
    void add<T>(args);
    T& get<T>();
}

// ends up being slow in practice due to implementation details
    - difficult to implement, must map Component class to index
```

4. Component Container

```
Entity e;
e.add<CTransform>(Vec2(100, 200));
e.add<CName>("Red Box");
e.add<CShape>(args);
if (e.has<CTransform>()) { ... }

// abstracts away storage of components details
   - allows storage changes without usage code changes
// components stored contiguously in memory
// least amount of code to type
```

5. Component Tuple

```
class Entity
{
    std::tuple<C1, C2, C3, C4> m_components;
public:
    Entity() {}
    void add<T>(args);
    T& get<T>();
    bool has<T>();
    void remove<T>();
}
// much faster and easier in practice than vector-based
```

5. Component Tuple

```
Entity e;
e.add<CTransform>(Vec2(100, 200));
e.add<CName>("Red Box");
e.add<CShape>(args);
if (e.has<CTransform>()) { ... }

// abstracts away storage of components details
   - allows storage changes without usage code changes
// tuple uses stack, but not guaranteed contiguous
// least amount of code to type
```

Additional Entity Variables

```
class Entity
{
    std::tuple<C1, C2, C3, C4> m_components;



public:
    Entity() {}
    void                        add<T>(args);
    T&                          get<T>();




}
```

Additional Entity Variables

```
class Entity
{
    std::tuple<C1, C2, C3, C4> m_components;
    bool                       m_alive = true;    // if false, will delete entity
    std::string                m_tag = "default"; // type of entity
    size_t                     m_id = 0;          // unique integer id
public:
    Entity() {}
    void                       add<T>(args);
    T&                         get<T>();
    size_t                     id() const;
    bool                       isAlive() const;
    void                       destroy();
    const std::string&         tag() const;
}
```

Convenient Syntax

```
using ComponentTuple = std::tuple<
    CTransform,
    CLifeSpan,
    CInput,
    CBoundingBox,
    CAnimation,
    CGravity,
    CState
>;
```

Additional Entity Variables

```
class Entity
{
    std::tuple<C1, C2, C3, C4> m_components;
    bool                       m_alive = true;    // if false, will delete entity
    std::string                m_tag = "default"; // type of entity
    size_t                     m_id = 0;          // unique integer id
public:
    Entity() {}
    void                       add<T>(args);
    T&                         get<T>();
    size_t                     id() const;
    bool                       isAlive() const;
    void                       destroy();
    const std::string&         tag() const;
}
```

Additional Entity Variables

```
class Entity
{
    ComponentTuple              m_components;
    bool                        m_alive = true;    // if false, will delete entity
    std::string                 m_tag = "default"; // type of entity
    size_t                      m_id = 0;          // unique integer id
public:
    Entity() {}
    void                        add<T>(args);
    T&                          get<T>();
    size_t                      id() const;
    bool                        isAlive() const;
    void                        destroy();
    const std::string&          tag() const;
}
```

Using std::tuple

```
// declare tuple of various types
std::tuple<int, double, int, float, size_t> myTuple;
// if type present only once, can std::get by type
std::get<double>(myTuple) = 3.14159;
// if not unique, you can specify the ‘index’
std::get<2>(myTuple) = 42;
// same syntax used to get values for use
std::cout << std::get<double>(myTuple) << "\n";
// can also initialize with different types
std::tuple<int, std::string, float> myTuple(42, "Hello", 3.14f);
```

Entity Functions: get

```
template <typename T>
T & get()
{
    return std::get<T>(m_components);
}
// const version to keep const correctness
template <typename T>
const T& get() const
{
    return std::get<T>(m_components);
}
```

Entity Functions: has / remove

```
template <typename T>
bool has() const
{
    return get<T>().exists;
}

template<typename T>
void remove()
{
    get<T>() = T();
}
```

Entity Functions: add

```
template <typename T, typename... TArgs>
T& add(TArgs&&... mArgs)
{
    auto & component = get<T>();
    component = T(std::forward<TArgs>(mArgs)...);
    component.exists = true;
    return component;
}
// component default constructor sets ‘has’ to false
// only set it to true when we explicitly add a component
```

Component Implementation

- We will have a different `Component` class for each Component we want to implement
- Naming: class `CComponentName`
- Components will inherit from a base `Component` class which stores any data we want to be common to all components

Component Base Class

```
class Component
{
public:
    bool exists = false;
};

// exists = whether entity actually contains
            the derived component
// default to false so default constructed components
   are not considered part of the Entity
```

ECS Component: CTransform

```
class CTransform : public Component
{
public:
    Vec2 pos = {0,0};
    Vec2 velocity = {0,0};
    CTransform() {}
    CTransform(const Vec2 & p, const Vec2 & v)
        : pos(p), velocity(v) {}
};
```

ECS Component: CShape

```
class CShape : public Component
{
public:
    sf::CircleShape shape;
    CShape() {}
};
```

ECS Component: CLifeSpan

```
class CLifeSpan : public Component
{
public:
    int totalLifespan = 0;
    int remainingLifespan = 0;
    CLifeSpan() {}
    CLifeSpan(int totalLifespan)
        : totalLifespan(totalLifespan)
        , remainingLifespan(totalLifespan) {}
};
```

Using Entities

```
void doStuff(std::vector<Entity> & entities)
{
    for (auto& e : entities)
    {
        e.cTransform.pos += e.cTransform.velocity;
        e.cShape.shape.setPosition(e.cTransform.pos);
        window.draw(e.cShape.shape);
    }
}
```

ECS: Systems

```
void sMovement(std::vector<Entity> & entities)
{
    for (auto& e : entities)
    {
        e.CTransform.pos += e.CTransform.velocity;
    }
}
```

ECS: Systems

- Some systems only operate on entities which contain certain Components
  - Movement: CTransform
  - Render: CTransform + CShape
  - Physics: CBoundingBox
- We can filter entities before passing them into system, or just check entities for required components before using them

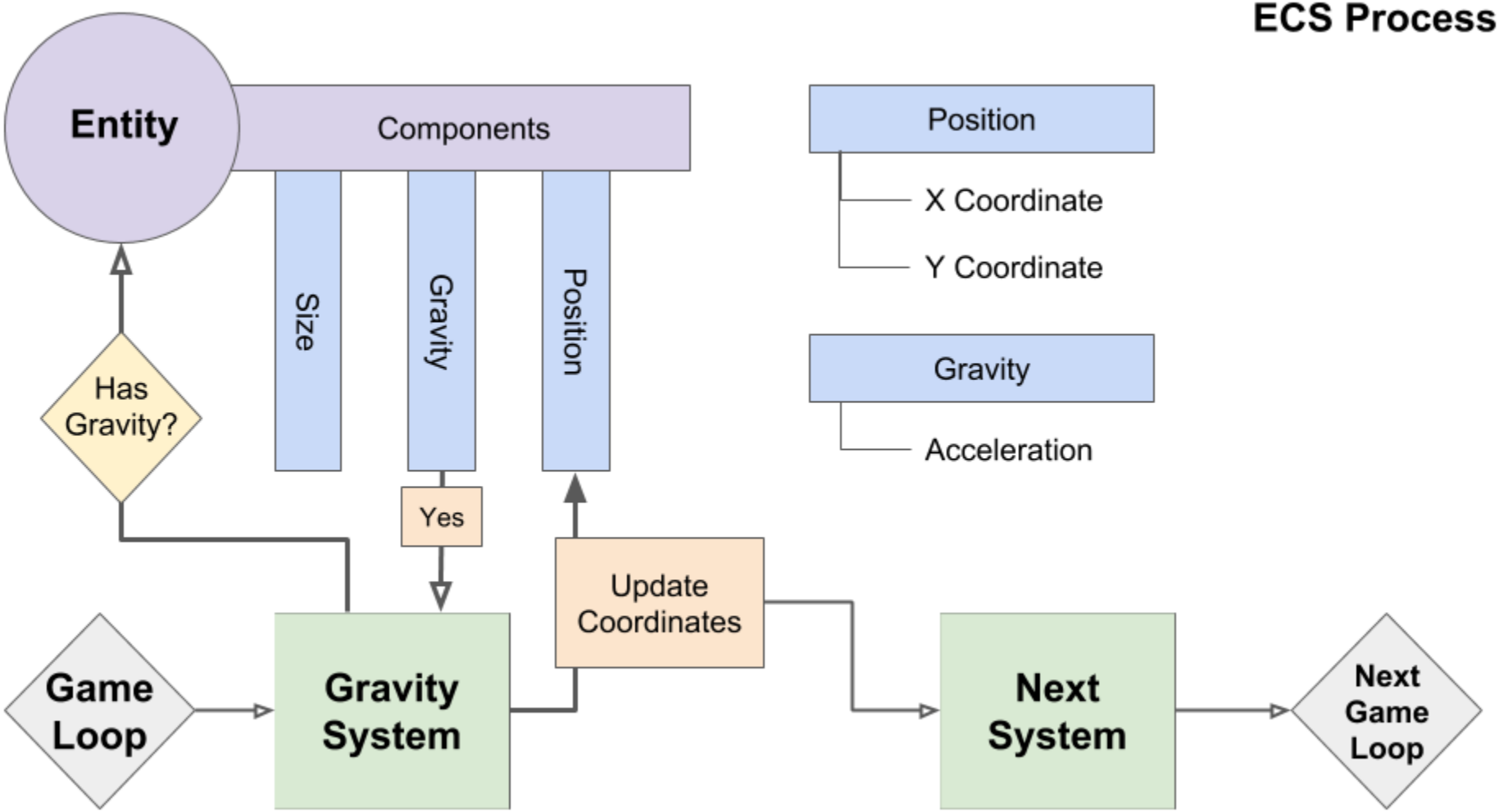
ECS: Systems

```
void sLifespan()
{
    for (auto& e : m_entityManager.getEntities())
    {
        if (!e->has<CLifeSpan>()) { continue; }
        e.get<CLifeSpan>().remainingLifespan--;
        if (e.get<CLifeSpan>().remainingLifespan <= 0)
        {
            e.destroy();
        }
    }
}
```

ECS: Systems

```
void sRender()
{
    for (auto& e : m_entityManager.getEntities())
    {
        if (e.has<CShape>() && e.has<CTransform>())
        {
            e.cShape.shape.setPosition(e.CTransform.pos);
            window.draw(e.cShape.shape);
        }
    }
}
```

ECS Process


Entity
Components
Size
Gravity
Position
Has Gravity?
Yes
Update Coordinates
Game Loop
Gravity System
Next System
Next Game Loop
Position
X Coordinate
Y Coordinate
Gravity
Acceleration

Vec2 Class

Vectors in Games

- For 2D games, many quantities / properties of game objects have both an X and a Y coordinate, these form a 2D Vector
  - Mathematical Vector, not collection vector
- Example: Position(x,y) and Velocity(x,y)
- Annoying to store / refer to xy separately
- Let's make a 2D Vector class: Vec2

Notes on Vec2

- We will go more in-depth on vector math for games later in the course
- We first introduce the programming architecture, then later use it to implement game math / physics concepts
- Note: sf::Vector2f already exists, we will basically be reinventing it in this course

Vec2 Class

- What do we want to store in Vec2?
  - Member variables x, y
- What do we want to do with those values?
  - Functions / Operators on the Vec2 class
- Possible Operations on Vec2
  - +, -, *, /
  - length, normalize, rotate, etc

Vec2 Class

```
class Vec2
{
public:
    float x = 0, y = 0;
    Vec2();
    Vec2(float xin, float yin);
    bool operator == (const Vec2 & rhs) const;
    Vec2 operator + (const Vec2 & rhs) const;
    Vec2 operator * (const float & val) const;
    void operator += (const Vec2 & rhs);
};
```

Vec2 Sample Usage

```
Vec2 pos     (100,  200);
Vec2 velocity( -5,   10);
Vec2 accel   (  0, -9.8);
while (true)
{
    pos += velocity;
    velocity += accel;
}
float dist = velocity.length();
```